



Ministério da Educação

Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Norte
CAMPUS SANTA CRUZ

Rua São Braz, 304, Paraíso, SANTA CRUZ / RN, CEP 59200-000

Fone: (84) 4005-4110

ATIVIDADE PRÁTICA REMOTA

Avaliação — Aula 01

Mural de Publicações 2.0: consumindo a API DummyJSON

Disciplina	Programação para Internet
Modalidade	Atividade prática remota
Prazo	1 semana
Realização	Individual, trio ou no máximo 4 integrantes
Valor	20,0 pontos
Tecnologias	HTML, CSS e JavaScript
API	DummyJSON — Posts

Desafio central: adaptar o mural desenvolvido na aula para uma nova API, interpretar corretamente a resposta e aprimorar a experiência do usuário.

1. Situação-problema

Na Aula 01, construímos um mural capaz de consultar publicações de uma API e apresentá-las dinamicamente no navegador. A primeira versão utilizou o JSONPlaceholder.

Agora, o projeto deverá ser adaptado para consumir a API DummyJSON. Embora as duas APIs disponibilizem publicações, elas apresentam estruturas de resposta diferentes. Portanto, não será suficiente substituir apenas o endereço da requisição.

Sua missão será desenvolver o Mural de Publicações 2.0, interpretando a nova resposta, selecionando os dados relevantes e aprimorando a experiência do usuário.

2. Questão norteadora

Como adaptar uma aplicação JavaScript para consumir uma nova API, interpretar sua estrutura e transformar os dados recebidos em uma interface interativa?

3. Objetivos da avaliação

- localizar e manipular elementos do DOM;
- organizar comportamentos por meio de funções;
- utilizar funções como callbacks;
- compreender operações assíncronas;
- utilizar `async`, `await` e `fetch()`;
- verificar a resposta HTTP e convertê-la para JSON;
- interpretar objetos, arrays e objetos aninhados;
- utilizar `slice()`, `map()`, `filter()`, `includes()` e `join()`;
- apresentar estados de carregamento, sucesso, erro e resultado vazio;
- explicar as decisões adotadas na solução;

4. Projeto inicial

Utilize como ponto de partida o projeto desenvolvido na Aula 01. A estrutura visual poderá ser preservada; a principal intervenção deverá ocorrer em `js/mural.js`. Não é necessário reconstruir integralmente o HTML e o CSS.

```
mural-duvidas/  
├─ index.html  
├─ css/  
│   └─ style.css  
└─ js/  
    └─ mural.js
```

5. Nova API

A aplicação deverá consultar o endereço abaixo:

```
https://dummyjson.com/posts
```

A resposta possui a estrutura geral a seguir. Observe que ela não é diretamente um array: o array de publicações encontra-se em `dados.posts`.

```
{  
  posts: [{  
    id: 1,  
    title: "Título da publicação",
```

```

    body: "Conteúdo da publicação",
    tags: ["história", "ficção"],
    reactions: { likes: 192, dislikes: 25 },
    views: 305,
    userId: 121
  }],
  total: 251,
  skip: 0,
  limit: 30
}

```

6. Requisitos obrigatórios

6.1 Consulta à DummyJSON

Utilize `fetch()` para consultar a API quando o usuário selecionar o botão “Carregar publicações”. Não serão aceitos dados escritos manualmente no JavaScript para substituir a resposta da API.

```
const resposta = await fetch("https://dummyjson.com/posts");
```

6.2 Programação assíncrona

A função de consulta deverá ser declarada com `async`, utilizar `await` na requisição e na conversão para JSON e possuir `try`, `catch` e `finally`.

```

async function carregarPublicacoes() {
  try {
    // consulta e processamento
  } catch (erro) {
    // tratamento da falha
  } finally {
    // finalização da operação
  }
}

```

6.3 Verificação da resposta HTTP

Verifique `resposta.ok`. Quando a resposta não estiver na faixa de sucesso, crie um erro com o status HTTP.

```

if (!resposta.ok) {
  throw new Error(`A API respondeu com o status ${resposta.status}.`);
}

```

6.4 Interpretação da nova estrutura

Depois da conversão, acesse corretamente o array por meio de `dados.posts`. `dados.slice(...)` estaria incorreto, pois `dados` é um objeto.

```

const dados = await resposta.json();
const publicacoes = dados.posts;

```

6.5 Seleção das publicações

Apresente inicialmente as 12 primeiras publicações e confirme a quantidade na interface com a mensagem “12 publicações carregadas.”

```
publicacoesCarregadas = dados.posts.slice(0, 12);
```

6.6 Cartões aprimorados

Cada cartão deverá apresentar identificador, título, conteúdo, tags, quantidade de curtidas e quantidade de visualizações. Use `publicacao.id`, `title`, `body`, `tags`, `reactions.likes` e `views`.

```
publicacao.tags.join(", ")
```

6.7 Geração dinâmica dos cartões

Construa os cartões por meio de uma função `criarCartao(publicacao)`, utilizando parâmetro, `return`, template string e propriedades do objeto. Transforme as publicações com `map()` e `join()`. Não crie manualmente 12 cartões no HTML.

```
publicacoes.map(criarCartao).join("");
```

6.8 Pesquisa

Permita pesquisar por título, conteúdo e tags, ignorando diferenças entre maiúsculas e minúsculas. Utilize `filter()` e `includes()`.

```
const texto = `${publicacao.title} ${publicacao.body} ${publicacao.tags.join(" ")}`.toLowerCase();
```

6.9 Estados da interface

Informe claramente os estados inicial, carregamento, sucesso, resultado da pesquisa, pesquisa sem resultado e falha.

Estado	Mensagem
Inicial	Clique em “Carregar publicações” para começar.
Carregamento	Carregando publicações...
Sucesso	12 publicações carregadas.
Pesquisa	3 publicações encontradas.
Sem resultado	Nenhuma publicação corresponde à pesquisa.
Falha	Não foi possível carregar os dados. Tente novamente.

6.10 Controle do botão e da pesquisa

Durante a requisição, mantenha botão e pesquisa desabilitados e `aria-busy="true"`. Ao final, reabilite o botão, altere `aria-busy` para `"false"` e habilite a pesquisa se os dados forem carregados.

6.11 Eventos

Registre os eventos de clique e digitação. Forneça as funções como callbacks, sem execução imediata.

```
botaoCarregar.addEventListener("click", carregarPublicacoes);
campoPesquisa.addEventListener("input", filtrarPublicacoes);
```

7. Requisito de autoria: explicação técnica

Além do projeto, inclua o arquivo `EXPLICACAO.md`, respondendo com palavras próprias:

1. Qual é a diferença entre a resposta do `JSONPlaceholder` e a resposta da `DummyJSON`?
2. Por que utilizamos `dados.posts`?
3. O que o primeiro `await` aguarda?
4. O que o segundo `await` aguarda?
5. Onde existe uma callback no projeto?
6. Como `map()` participa da criação dos cartões?
7. Como `filter()` participa da pesquisa?
8. O que acontece quando a requisição falha?
9. Qual foi a maior dificuldade encontrada?
10. Que alteração foi feita além da adaptação mínima?

Atenção: respostas excessivamente genéricas ou que não correspondam ao código entregue não serão consideradas suficientes.

8. Personalização obrigatória

Implemente pelo menos uma personalização funcional, para que a atividade não se limite à reprodução do projeto da aula:

- botão para limpar a pesquisa;
- ordenação por quantidade de curtidas;
- ordenação por quantidade de visualizações;
- filtro por tag;
- destaque para publicações com mais de determinada quantidade de curtidas;
- contador do total de curtidas dos cartões exibidos;
- botão para recarregar os dados;

- alternância entre seis e 12 publicações;
- apresentação da quantidade de descurtidas;
- mensagem personalizada para pesquisa sem resultado;

A personalização deverá ser explicada no arquivo EXPLICACAO.md.

9. Resultado mínimo esperado

Ao abrir o projeto

- título do mural;
- botão de carregamento;
- campo de pesquisa desabilitado;
- mensagem inicial;
- área de cartões vazia;

Depois do carregamento

- 12 cartões apresentados;
- dados obrigatórios em cada cartão;
- pesquisa habilitada;
- mensagem com a quantidade carregada;
- console sem erros;

Durante a pesquisa

- cartões atualizados;
- título, conteúdo e tags considerados;
- quantidade encontrada informada;
- mensagem específica sem correspondências;

10. Estrutura da entrega

```
nome-sobrenome-mural-dummyjson/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── mural.js  
├── EXPLICACAO.md  
└── README.md
```

Conteúdo do README.md

```
# Mural de Publicações 2.0

## Estudantes
Nome completo:

## Turma
Turma:

## Objetivo
Breve descrição do projeto.

## Como executar
Orientações para abrir e testar a aplicação.

## API utilizada
https://dummyjson.com/posts

## Personalização implementada
Descrição da funcionalidade adicional.
```

11. Forma de envio

1. Organize os arquivos na estrutura solicitada.
2. Teste o projeto antes do envio.
3. Compacte a pasta no formato .zip.
4. Nomeie o arquivo como **nomesdosintegrantes-mural-dummyjson.zip**.
5. Submeta o projeto como resposta da atividade no Class.
6. Confirme que o arquivo enviado pode ser aberto.

Não deverão ser enviados somente o arquivo mural.js, capturas de tela, links inacessíveis ou códigos colados no campo de comentários.

12. Cronograma sugerido para a semana

Período	Atividade
Dia 1	Executar o projeto inicial e examinar a DummyJSON
Dia 2	Adaptar fetch() e interpretar dados.posts
Dia 3	Construir os cartões aprimorados
Dia 4	Implementar a pesquisa
Dia 5	Implementar estados e tratamento de erros
Dia 6	Criar a personalização e escrever a explicação
Dia 7	Testar, revisar, compactar e enviar

O cronograma é uma sugestão. O estudante poderá organizar o trabalho de outra maneira, desde que respeite o prazo final.

13. Critérios de avaliação

Critério	Pontos
Consulta correta à DummyJSON e interpretação de dados.posts	3,0
Uso adequado de async, await, fetch() e conversão JSON	3,0
Verificação HTTP e tratamento com try, catch e finally	2,0
Criação dinâmica dos cartões com os dados obrigatórios	3,0
Uso de arrays, objetos, slice(), map() e join()	2,0
Pesquisa por título, conteúdo e tags	2,0
Estados da interface e controle dos elementos	1,0
Personalização funcional	2,0
Organização, legibilidade e funcionamento do código	1,0
README.md e EXPLICACAO.md	1,0
TOTAL	20,0

14. Faixas de desempenho

Faixa	Descrição
19,0 – 20,0	Funciona integralmente, interpreta a API, apresenta código organizado, personalização funcional e explicação consistente.
13,0 – 18,9	Atende aos requisitos principais, com pequenas limitações na pesquisa, nos estados ou na explicação.
7,0 – 12,9	O consumo da API funciona parcialmente, mas há problemas de apresentação, filtragem, organização ou compreensão.
3,0 – 6,9	Apresenta execução limitada, reprodução incompleta ou dificuldades relevantes na interpretação da resposta.
0 – 2,9	Não executa, não consome a API solicitada, está incompleto ou não evidencia autoria suficiente.

15. Checklist do estudante

- ☐ O projeto utiliza a DummyJSON.
- ☐ O endereço da API está correto.

- ☐ O código acessa dados.posts.
- ☐ A página apresenta 12 publicações.
- ☐ Os cartões possuem título e conteúdo.
- ☐ As tags, curtidas e visualizações são apresentadas.
- ☐ A pesquisa considera título, conteúdo e tags.
- ☐ Existem mensagens de carregamento, sucesso, erro e pesquisa sem resultado.
- ☐ O botão é reabilitado ao final.
- ☐ Não existem erros no console.
- ☐ A personalização funciona.
- ☐ README.md e EXPLICACAO.md estão preenchidos.
- ☐ A pasta foi compactada corretamente.
- ☐ O arquivo enviado abre e executa.

16. Orientação sobre consulta e autoria

Será permitida a consulta ao material da aula, à documentação da DummyJSON e do JavaScript, às anotações pessoais e aos exemplos produzidos durante as práticas. A consulta deve apoiar a aprendizagem, e não substituir a compreensão.

O estudante deverá ser capaz de explicar o código entregue. Soluções incompatíveis com as respostas do EXPLICACAO.md, cópias integrais sem adaptação ou códigos que o estudante não consiga justificar poderão ser submetidos a uma verificação complementar.

Comando final para os estudantes

Adapte o Mural de Dúvidas desenvolvido em aula para consumir a API DummyJSON. Interprete corretamente a estrutura dados.posts, apresente 12 publicações com informações ampliadas, implemente pesquisa por título, conteúdo e tags, trate os diferentes estados da aplicação e acrescente uma personalização funcional. Teste a solução, registre sua explicação técnica e envie o projeto completo no prazo de uma semana.